

AI and Intelligent Transport System (ITS)

ฉบับอ่านเข้าใจง่าย พร้อมสอบและฟรีเชนต์

สรุปจากสไลด์ VAI(1).pdf - 18 May 2026

Architectural Case Studies



“Passive Sensing” Concept in ITS

- M-PASS
- Easy-PASS
- Intelligent Parking System



“Privacy” Concept in ITS

- Edge Nodes (NVIDIA Jetson)
- Edge Computing
- Federated Approach



“Centralized Decision Data”

- Data Aggregation on Cloud Server
- Dashboard & Traffic Heatmaps
- Automated Govt Services (Tickets/Reports)

รูปจากสไลด์หน้า 8: ภาพรวมแนวคิด Passive Sensing, Privacy/Edge และ Centralized Decision Data

เป้าหมายของเอกสารนี้

อ่านเพื่อเข้าใจภาพรวม AI Vision ใน ITS, Object Detection, YOLO, RT-DETR, Edge Deployment และ Vehicle ReID โดยเรียงตามลำดับเนื้อหาในสไลด์ พร้อมตารางสรุป คำศัพท์ และแนวข้อสอบท้ายเล่ม

สารบัญ

1. บทนำ: เอกสารนี้เกี่ยวกับอะไร
2. Chapter 1: ภาพรวม AI Vision และ ITS
3. Chapter 2: Edge Computing สำหรับ AI Vision
4. Chapter 3: Object Detection ในระบบ ITS
5. Chapter 4: YOLO แบบเข้าใจง่าย
6. Chapter 5: RT-DETR และ Transformer สำหรับ Object Detection
7. Chapter 6: Evaluation Metrics
8. Chapter 7: Implementation & Deployment
9. Chapter 8: Vehicle Re-Identification (Vehicle ReID)
10. Chapter 9: Transformer Architecture for ReID
11. Chapter 10: Vehicle ReID Datasets
12. Chapter 11: Fast ReID Framework
13. Chapter 12: Pre-processing และ Augmentation
14. Chapter 13: Feature Extraction, Aggregation, Head และ Loss
15. Chapter 14: City Scale ReID และ Spatial-Temporal Integration
16. สรุปภาพรวมทั้งหมด + กล้องจำง่าย
17. Glossary คำศัพท์สำคัญ
18. คำถามทบทวน + แนวข้อสอบพร้อมเฉลย

บทนำ: เอกสารนี้เกี่ยวกับอะไร

เอกสารนี้สรุปเนื้อหาจากสไลด์เรื่อง AI and Intelligent Transport System (ITS) โดยแปลงให้อยู่ในรูปแบบหนังสือเรียนภาษาไทยที่อ่านง่าย เหมาะสำหรับผู้เริ่มต้นที่ยังไม่คุ้นกับ AI Vision, Edge Computing, Object Detection และ Vehicle Re-identification มาก่อน

แกนหลักของสไลด์แบ่งเป็น 2 ส่วนใหญ่ คือ Part 1: AI Vision in ITS Paradigm และ Part 2: Advanced Vehicle Re-identification (ReID)

เอกสารนี้จะเรียงเนื้อหาตามลำดับจากภาพรวมระบบขนส่งอัจฉริยะ ไปสู่โมเดลตรวจจับวัตถุ การ deploy บนอุปกรณ์ Edge และการจดจำรถคันเดิมซ้ำมกล้อง

วิธีอ่านให้พร้อมสอบ

ให้จำภาพใหญ่ก่อน: ITS ต้องการข้อมูลจากถนน -> กล้องกลายเป็น Sensor -> Edge ช่วยลด Bandwidth -> Object Detection
ตรวจจับรถ -> ReID จดจำรถคันเดิมซ้ำมกล้อง -> Spatial/Temporal data ช่วยเชื่อมโยงเหตุการณ์ระดับเมือง

Chapter 1: ภาพรวม AI Vision และ ITS

ITS หรือ Intelligent Transport System คือระบบขนส่งอัจฉริยะที่ใช้ข้อมูลจากถนน ยานพาหนะ และโครงสร้างพื้นฐาน เพื่อช่วยบริหารจัดการจราจร ตรวจจับเหตุการณ์ ลดความแออัด และสนับสนุนบริการภาครัฐ เช่น ค่าผ่านทาง รายงานอุบัติเหตุ หรือระบบจอดรถอัจฉริยะ

1.1 Bottleneck ของ ITS แบบเดิม

สไลด์เริ่มจากปัญหาของ Legacy ITS technologies ซึ่งพึ่งพา sensor-based devices เป็นหลัก ระบบแบบเดิมใช้ bandwidth ต่ำและสื่อสารแบบ multi-hop ได้ แต่ภาพหรือวิดีโอไม่ได้เป็นข้อมูลหลักในการประมวลผล ทำให้ระบบรู้ข้อมูลเชิงลึกของวัตถุบนถนนได้น้อย

- Infrared sensor ต้องติดตั้งจำนวนมาก ทำให้ต้นทุน deployment สูง
- อุปกรณ์ sensor แบบเดิมมักบอกได้แค่สัญญาณหรือสถานะบางอย่าง แต่ไม่เห็นรายละเอียดของวัตถุ
- การวิเคราะห์เชิงลึก เช่น ประเภทรถ สีรถ ทิศทาง และการระบุตัวตนของวัตถุ ทำได้จำกัด

1.2 การเปลี่ยนผ่านสู่ Vision-AI

แนวคิดสำคัญคือกล้องราคาถูกลงและเข้าถึงง่ายขึ้น จึงสามารถมองกล้องเป็น Sensor ได้ แต่เป็น sensor ที่ให้ข้อมูลหลากหลายกว่าเดิมมาก กล้อง 1 ตัวสามารถใช้แทน sensor หลายรูปแบบ เพราะภาพสามารถนำไปทำ Object Detection, Object Recognition และ Object Identification ได้

- Automated Plate Recognition: อ่านป้ายทะเบียนอัตโนมัติ
- Congestion Monitoring: ตรวจสอบความหนาแน่นหรือความติดขัดของถนน
- Intelligent Parking: ตรวจสอบที่จอดรถว่างและสถานะการจอด

Architectural Case Studies



“Passive Sensing” Concept in ITS

- M-PASS
- Easy-PASS
- Intelligent Parking System



“Privacy” Concept in ITS

- Edge Nodes (NVIDIA Jetson)
- Edge Computing
- Federated Approach



“Centralized Decision Data”

- Data Aggregation on Cloud Server
- Dashboard & Traffic Heatmaps
- Automated Govt Services (Tickets/Reports)

รูปจากสไลด์หน้า 8: กรอบสถาปัตยกรรม ITS ที่เชื่อม Passive Sensing, Privacy/Edge และ Centralized Decision Data

ง่าย

กล้อง 1 ตัวไม่ได้เป็นแค่กล้อง แต่เป็น Sensor หลายแบบในตัวเดียว เพราะสามารถตรวจจับ จำแนก และระบุตัวตนของวัตถุได้

Chapter 2: Edge Computing สำหรับ AI Vision

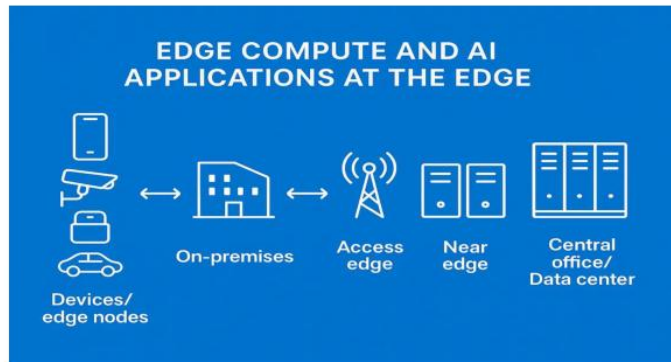
2.1 ทำไมส่งวิดีโอทั้งหมดขึ้น Cloud จึงเป็นปัญหา

Modern ITS ต้องใช้ภาพความละเอียดสูงสำหรับงาน classification และ Re-identification เช่น 4K, 30 FPS และ codec H.265 โดยสไลด์ให้ค่า bitrate ของกล้อง 4K + 30 FPS ประมาณ 8 Mbps ต่อกล้อง

ถ้ามี 100 กล้อง จะใช้ uplink ประมาณ 800 Mbps ซึ่งเกือบชนขีดจำกัดของ fiber optic uplink 1 Gbps ในสถานการณ์จริง และถ้าขยายเป็น 1,000 กล้องระดับเมือง จะต้องใช้ประมาณ 8 Gbps ซึ่งสไลด์ระบุว่า infeasible และแพงต่อการ deployment

Edge Computing

→ Distributing Edge Compute Nodes to Data Sources ACAP (As Closed As Possible)



<https://medium.com/@PhaniBhushanAthlur/top-edge-computing-platforms-compared-challenges-trends-and-how-to-use-ai-at-the-edge-514358d0ba8b>

รูปจากสไลด์หน้า 11: Edge Computing คือการวาง compute node ใกล้ data source ให้มากที่สุด

2.2 แนวคิด Edge Computing

Edge Computing คือการนำการประมวลผลไปไว้ใกล้แหล่งข้อมูล เช่น ใกล้กล้อง CCTV หรือในอุปกรณ์ภาคสนาม แทนที่จะส่งวิดีโอดิบทั้งหมดไป Cloud เมื่อประมวลผลที่ edge แล้ว ระบบส่งเฉพาะข้อมูลจำเป็น เช่น JSON metadata และภาพยืนยันขนาดเล็ก

- จากวิดีโอ 8 Mbps ต่อกำลัง ลดเหลือข้อมูลประมาณ < 2KB JSON + < 4KB Image
- Edge node ต้องมี hardware สำหรับ AI inference และโมเดล Vehicle Object Detection
- ข้อมูลที่ส่งออกมาจะเป็น “Vehicle, White Toyota Vios, arrives at timestamp, exits at timestamp”

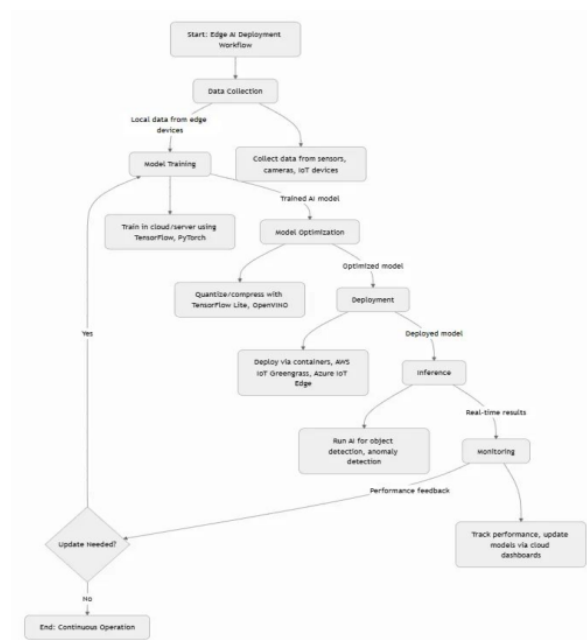
ประเด็น	Edge Computing	Cloud Computing
Latency	ต่ำมาก ตอบสนองใกล้แหล่งข้อมูล เช่น < 20 ms	สูงกว่า ขึ้นกับ network เช่น 100-300 ms
Bandwidth	ส่งเฉพาะ metadata เช่น < 2KB JSON	ต้องส่ง raw video เช่น 8 Mbps ต่อกำลัง 4K
Processing Power	จำกัดกว่า ต้องใช้โมเดล optimize	ขยายพลังประมวลผลได้มาก ใช้โมเดลใหญ่ได้
Network Resilience	ทนต่อปัญหา network มากกว่า กู้ข้อมูลง่ายกว่า	ถ้าข้อมูลวิดีโอใหญ่สูญหายจะ recovery ยากกว่า

2.3 Edge Deployment Workflow

- Data Collection: เก็บข้อมูลภาพหรือวิดีโอจากกล้อง
- Model Training: train โมเดลสำหรับงานที่ต้องการ
- Model Optimization: ลดขนาดหรือ optimize ให้เหมาะกับ edge device
- Model Deployment: นำโมเดลไปติดตั้งบนอุปกรณ์
- Model Inference: รันตรวจจับจริง
- Performance Monitoring: ตรวจ latency, accuracy, error
- Update Phase: อัปเดตโมเดลเมื่อสภาพแวดล้อมเปลี่ยน

Edge Deployment Workflow

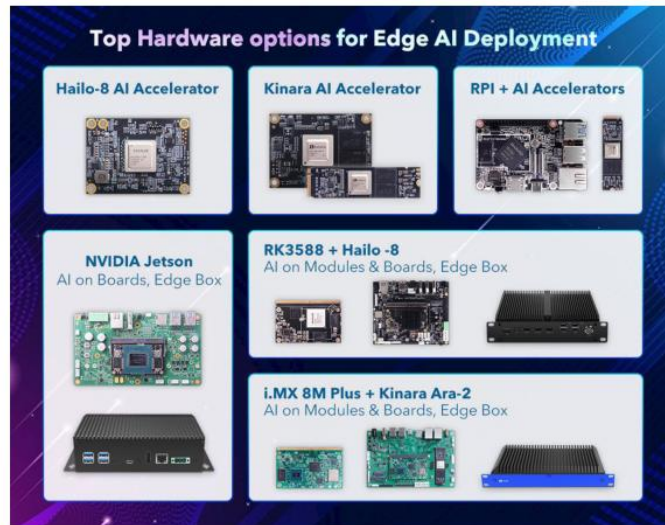
- Data Collection
- Model Training
- Model Optimization
- Model Deployment
- Model Inference
- Performance Monitoring
- Update Phase*



รูปจากสไลด์หน้า 14: Workflow ตั้งแต่เก็บข้อมูล train optimize deploy inference monitoring และ update

Edge AI Hardware Examples

- Hailo-8 AI Accelerator
- Kinara AI Accelerator
- RPI + AI Accelerator
- RK3588 + Hailo-8
- Kinara Ara-2 + MX8M+
- NVIDIA Jetson



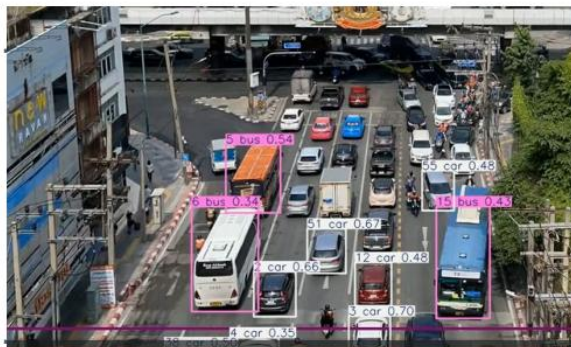
<https://www.geniatech.com/ai-hardware-2025/>

รูปจากสไลด์หน้า 15: ตัวอย่าง hardware สำหรับ Edge AI เช่น Hailo, Kinara, RPI + AI Accelerator, RK3588 และ NVIDIA Jetson

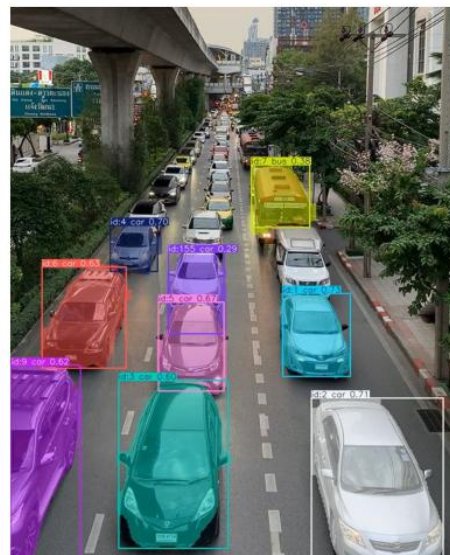
Chapter 3: Object Detection ในระบบ ITS

Object Detection คือภารกิจที่โมเดลต้องตอบ 2 คำถามพร้อมกัน คือ “วัตถุอยู่ตรงไหน” และ “วัตถุนั้นคืออะไร” ในระบบ ITS วัตถุอาจเป็นรถยนต์ รถเมล์ รถจักรยานยนต์ คนเดินถนน หรือวัตถุอื่นบนถนน

Object Detection Task in ITS



- Two sub-problem :
- Where is the vehicle ? (Boundary Box)
- What is the object ? (Classification)



รูปจากสไลด์หน้า 18: ตัวอย่าง Object Detection บนภาพถนน พร้อม bounding box และ class ของรถแต่ละคัน

ตัวอย่างง่าย

ถ้ากล้อง CCTV เห็นรถบนถนน โมเดลต้องตอบ 2 อย่าง: รถอยู่ตรงไหน = Bounding Box และรถเป็นประเภทอะไร = Classification

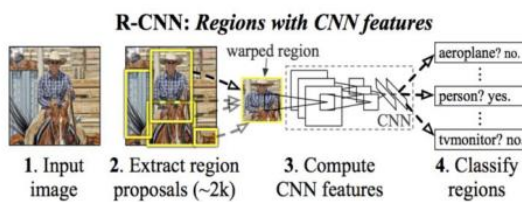
3.1 Two-Stage Detector

Two-Stage Detector เช่น Fast R-CNN / Faster R-CNN แบ่งงานเป็น 2 ขั้นตอน คือหา region ที่น่าจะมีวัตถุก่อน แล้วค่อย extract feature และ classify ว่าวัตถุนั้นเป็นคลาสอะไร

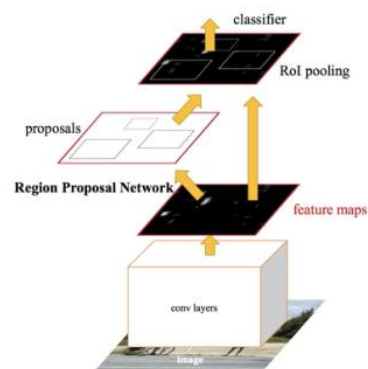
- Region Proposal Network (RPN): ถามว่า “Where are the objects?”
- Feature Extraction and Classification: ถามว่า “Which class should this object be in?”

Fast R-CNN

→ Original R-CNN



→ Faster R-CNN



รูปจากสไลด์หน้า 21: การพัฒนาจาก R-CNN ไป Faster R-CNN และบทบาทของ RPN

3.2 One-Stage Detector

One-Stage Detector เช่น YOLO ใช้วิธีรวมทุกอย่างไว้ในขั้นตอนเดียว โดยแบ่งภาพเป็น grid แล้วให้โมเดลทำนาย bounding box, confidence score และ class probability พร้อมกัน วิธีนี้ง่ายกว่าและเร็วกว่า จึงเหมาะกับงาน real-time

ประเภทโมเดล	แนวคิดหลัก	จุดเด่น	ตัวอย่าง
Two-Stage Detector	หา region ก่อน แล้ว classify	แม่นยำดีในหลายสถานการณ์ แต่ช้ากว่า	Fast R-CNN, Faster R-CNN
One-Stage Detector	ทำนายตำแหน่งและคลาสพร้อมกัน	เร็ว เหมาะกับ real-time	YOLO

Chapter 4: YOLO แบบเข้าใจง่าย

YOLO ย่อมาจาก You Only Look Once เป็น One-Stage Detector ที่ใช้ neural network เดียวทำนาย bounding boxes และ class probabilities พร้อมกัน จุดเด่นคือความเร็วมาก เหมาะกับงาน real-time object detection

YOLO Architecture

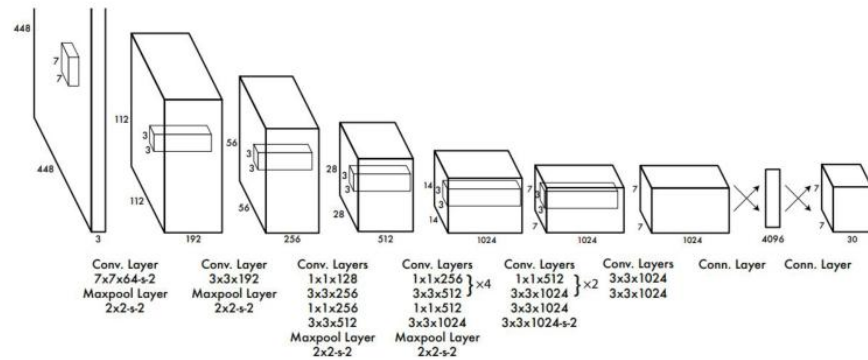


Figure 3: The Architecture. Our detection network has 24 convolutional layers followed by 2 fully connected layers. Alternating 1×1 convolutional layers reduce the features space from preceding layers. We pretrain the convolutional layers on the ImageNet classification task at half the resolution (224×224 input image) and then double the resolution for detection.

→ You Only Look Once: Unified, Real-Time Object Detection, Joseph Redmon*, Santosh Divvala*†, Ross Girshick¶, Ali Farhadi*†

รูปจากสไลด์หน้า 24: โครงสร้าง YOLO ดั้งเดิมที่ใช้ convolutional layers และ fully connected layers

4.1 ทำไม YOLO ถึงนิยม

- Speed: ทำงานเร็วมาก
- Detection Accuracy: ความแม่นยำดีเมื่อเทียบกับความเร็ว
- Generalization: ใช้งานได้ robust ในหลายภาพ
- Open Source: มีเครื่องมือและ community ให้ใช้งานง่าย

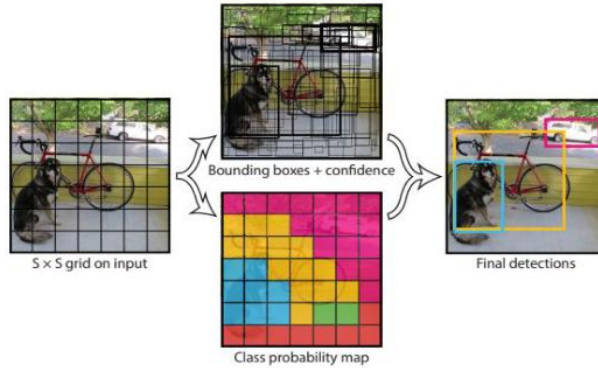
4.2 YOLO ทำงานอย่างไร

แนวคิดของ YOLO คือแบ่งภาพเป็น grid cells แต่ละ grid ช่วยทำนายกล่องและความน่าจะเป็นของคลาส จากนั้นระบบรวมคะแนนเพื่อเลือก detection ที่ดีที่สุด

- แบ่งภาพเป็น $N \times N$ หรือ $S \times S$ grid
- แต่ละ grid ทำนาย Bounding Box + Confidence Score
- ทำนาย Class Probability เช่น car 80%
- ผลลัพธ์สุดท้ายคำนวณจาก $\Pr(\text{Class} | \text{Object}) * \Pr(\text{Object}) * \text{IoU}$

How does YOLO work?

- Each Grid “predict” =>
 - ◆ B BoundingBox + Confidence Score
 - ◆ $\Pr(\text{Object}) * \text{IoU}$
- Each Grid “predict” =>
 - ◆ $\Pr(\text{Class}_i | \text{Object})$
- Final Detection =>
 - ◆ $\Pr(\text{Class}_i | \text{Object}) * \Pr(\text{Object}) * \text{IoU}$

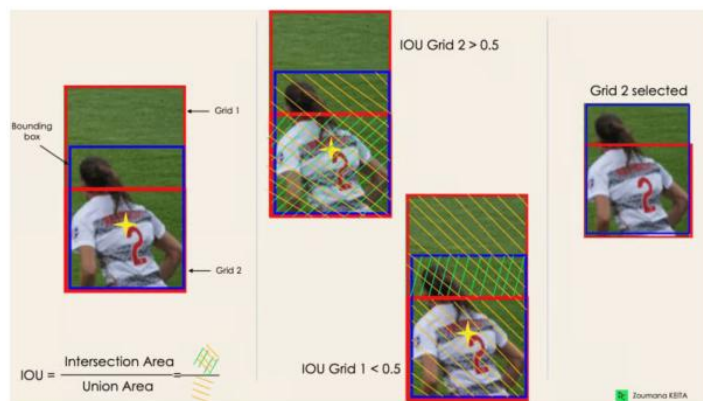


→ You Only Look Once: Unified, Real-Time Object Detection , Joseph Redmon*, Santosh Divvala*†, Ross Girshick†, Ali Farhadi*†

รูปจากสไลด์หน้า 27: YOLO ใช้ grid, bounding boxes, confidence และ class probability map เพื่อหา final detections

How does YOLO work?

- IoU Threshold
 - ◆ Use to define how high relevant Grid Cell should be to detect the object.



→ <https://www.datacamp.com/blog/yolo-object-detection-explained>

รูปจากสไลด์หน้า 31: IoU Threshold ใช้ตัดสินว่ากล่องทำนายซ้อนทับกับ ground truth เพียงพอหรือไม่

4.3 NMS และ Bottleneck

Non-Max Suppression หรือ NMS ใช้ลบ bounding boxes ที่ซ้ำซ้อนกันออก เหลือเฉพาะกล่องที่ดีที่สุด แต่เมื่อมีวัตถุเยอะและกล่องทับซ้อนจำนวนมาก NMS อาจกลายเป็น bottleneck

Non-Max Suppression or NMS

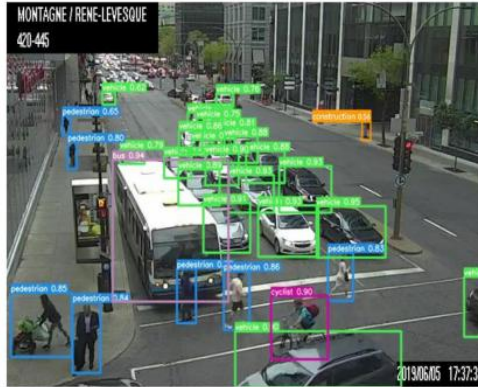
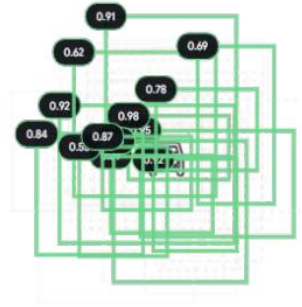


Figure From : An AI-Based Traffic Light Control System Using YOLOv10 with Emergency and Public Bus Prioritization: A Case Study in Baghdad



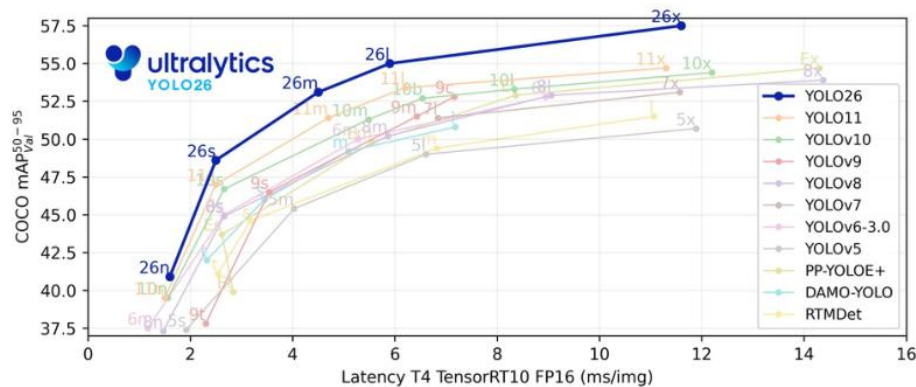
Green = High | Red = Overlap Suppressed | Grey = Low Confidence

→ CCTV => GPU (YOLO Inference 5ms) <mismatched> CPU (NMS > 10 ms) => Final Detection

รูปจากสไลด์หน้า 32: ตัวอย่าง bounding boxes จำนวนมากและขั้นตอน NMS

- ถ้าถนนว่าง มี 0 cars, NMS < 1 ms
- ถ้าเป็นแยกโศกเวลา 6PM อาจมี 150 รถ/มอเตอร์ไซค์ซ้อนกัน และ candidate boxes มากกว่า 1000
- กรณีซับซ้อน NMS อาจเกิน 30 ms ทำให้ performance ลดลง
- สไลด์ระบุว่า YOLOv10, v11, v12 เริ่มแก้ bottleneck ด้วย NMS-Free architecture

YOLO performance

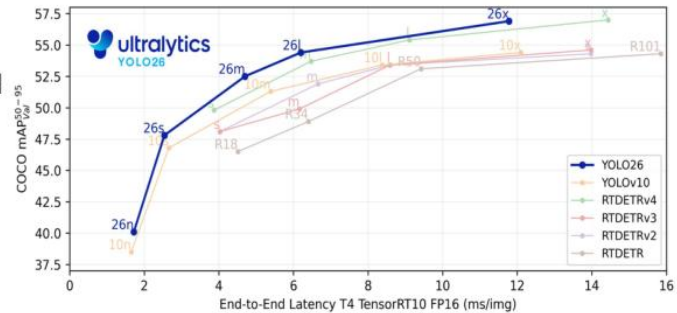


<https://docs.ultralytics.com/models/yolo26>

รูปจากสไลด์หน้า 35: กราฟ performance ของ YOLO รุ่นใหม่เทียบ latency และ mAP

YOLO 2026 (<https://docs.ultralytics.com/models/yolo26>)

- Native End-to-End Model
- NMS-Free Model (First solved in YOLO10 by Ao Wang at Tsinghua University)
- MuSGD Optimizer (Hybrid between SGD and Muon)
- 43% Faster on CPU
- ROC better than RT-DETR (Real-Time Detection Transformer)



รูปจากสไลด์หน้า 36: YOLO 2026: Native End-to-End, NMS-Free, MuSGD และเร็วขึ้นบน CPU

Chapter 5: RT-DETR และ Transformer สำหรับ Object Detection

RT-DETR คือ Real-Time variant ของ DETR หรือ Detection Transformer เป็นโมเดล object detection ที่ใช้แนวคิด Transformer โดยจุดเด่นคือ capture global context ได้ดีและเป็น NMS-Free

RT-DETR

- Real-Time Variant of DETR (DeTectioN TRansformer) based model.

.08069v3 [cs.CV] 3 Apr 2024

DETRs Beat YOLOs on Real-time Object Detection

Yian Zhao^{1,2†} Wenyu Lv^{1††} Shangliang Xu¹ Jinman Wei¹ Guanzhong Wang¹
Qingqing Dang¹ Yi Liu¹ Jie Chen^{2,✉}

¹Baidu Inc, Beijing, China ²School of Electronic and Computer Engineering, Peking University, Shenzhen, China
zhaoyian@stu.pku.edu.cn lvwenyu01@baidu.com jiechen2019@pku.edu.cn

Abstract

The YOLO series has become the most popular framework for real-time object detection due to its reasonable trade-off between speed and accuracy. However, we observe that the speed and accuracy of YOLOs are negatively affected by the NMS. Recently, end-to-end Transformer-based detectors (DETRs) have provided an alternative to eliminating NMS. Nevertheless, the high computational cost limits their practicality and hinders them from fully exploiting the advantage of excluding NMS. In this paper, we propose the **Real-Time DETECTION TRansformer (RT-DETR)**, the first real-time end-to-end object detector to our best knowledge that addresses the above dilemma. We build RT-DETR in two steps, drawing on the advanced DETR: first we focus on maintaining accuracy while improving speed, followed by maintaining speed while improving accuracy. Specifically, we design an efficient hybrid encoder to expeditiously process multi-scale features by decoupling intra-scale interaction and cross-scale fusion to improve speed. Then, we propose the uncertainty-minimal query selection to provide

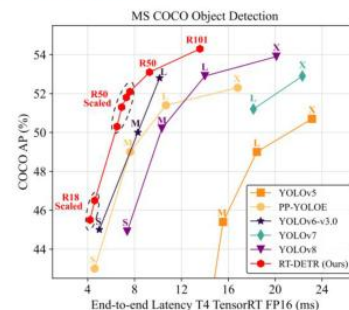


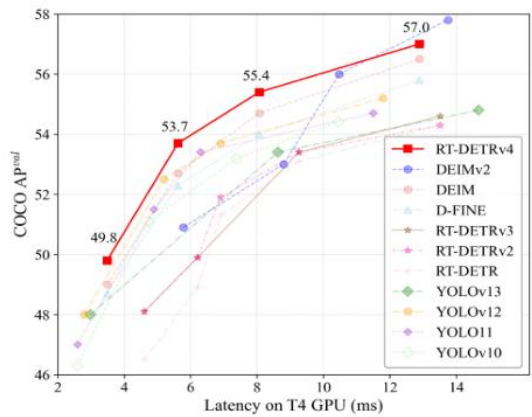
Figure 1. Compared to previously advanced real-time object detectors, our RT-DETR achieves state-of-the-art performance.

to their reasonable trade-off between speed and accuracy.

รูปจากสไลด์หน้า 37: RT-DETR เป็น real-time variant ของ DETR และเปรียบเทียบกับ YOLO ใน real-time detection

RT-DETRv4

→ <https://github.com/RT-DETRs/RT-DETRv4>



รูปจากสไลด์หน้า 38: RT-DETRv4 แสดงความสัมพันธ์ระหว่าง latency และ COCO AP

5.1 ทำไม Transformer น่าสนใจใน ITS

YOLO เป็น CNN-based model ที่เก่ง local feature extraction คือจับรายละเอียดเฉพาะบริเวณได้ดี เช่น รูปทรงรถหรือป้ายทะเบียน แต่ Transformer ใช้ self-attention จึงมองบริบททั้งภาพได้พร้อมกัน เหมาะกับฉากซับซ้อนและวัตถุซ้อนกันมาก

Project Constraint	YOLOv10/v12 (CNN / Local)	RT-DETR (Transformer / Global)
Best Use Case	Highways, Toll Booths, Parking Lots	Complex Urban Intersections, Roundabouts
Primary Strength	Raw FPS, localization สำหรับ license plate	Severe occlusion และ contextual reasoning
Primary Weakness	ลำบากเมื่อ target ถูกบังหนัก	ต้องใช้ memory bandwidth สูงกว่า
Hardware Target	Jetson Nano / Orin NX (15W-25W)	Jetson AGX Orin / Thor (40W-60W+)
Latency Profile	Sub-10ms ultra-low	15ms-25ms moderate

เลือกโมเดลแบบไหนดี

ถ้างานต้องเร็วมากและฉากไม่ซับซ้อน เช่น highway, toll booth, parking lot ให้เริ่มที่ YOLO แต่ถ้าเป็นแยกเมืองใหญ่ วัตถุซ้อนกันเยอะและต้องเข้าใจบริบททั้งภาพ ให้พิจารณา RT-DETR

Chapter 6: Evaluation Metrics

Metrics ใช้บอกว่าโมเดลตรวจจับได้ดีแค่ไหน ไม่ใช่ดูแค่เร็วอย่างเดียว ในงาน ITS ถ้าโมเดลพลาดจริงหรือแจ้งเตือนผิดมากเกินไป จะกระทบต่อระบบจราจรและการตัดสินใจ

Evaluation Metrics

- AP (Accurate Precision) :
 - ◆ Area Under the Precision-Recall curve for 1 Class*
- mAP (mean Accurate Precision) :
 - ◆ Area Under the Precision-Recall curves for all Classes*
- Precision
 - ◆ $\text{Precision} = \text{True Positive} / \text{Total Predictions}$
 - ◆ $\text{Total Predictions} = (\text{True Positive} + \text{False Positive})$
 - ◆ False Positive means "False alarm"
- Recall
 - ◆ $\text{Recall} = \text{True Positive} / \text{Total Ground Truths}$
 - ◆ $\text{Total Ground Truth} = (\text{True Positive} + \text{True Negative})$
 - ◆ True Negative means "miss target"

Where "TruePositive" is defined by IoU ≥ 0.5 (or higher)

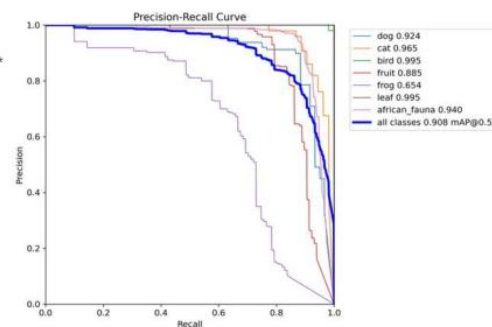


Figure from EDICA: A Hybrid Ensemble Architecture Using Deep Learning Models for Fine-Grained Image Classification

รูปจากสไลด์หน้า 42: Precision-Recall curve และนิยาม AP, mAP, Precision, Recall จากสไลด์

Metric	ความหมายแบบเข้าใจง่าย	ใช้ตีความอย่างไร
Precision	จากสิ่งที่โมเดลทำนายว่าใช่ มีถูกกี่ส่วน	ต่ำ = false alarm เยอะ
Recall	จากของจริงทั้งหมด โมเดลจับได้กี่ส่วน	ต่ำ = miss target เยอะ
AP	พื้นที่ใต้ Precision-Recall curve สำหรับ 1 class	วัดคุณภาพของคลาสเดียว
mAP	ค่าเฉลี่ย AP ทุก class	ภาพรวมของโมเดลหลายคลาส
mAP@50	คำนวณที่ IoU threshold 0.5	ไม่เข้มงวดเรื่องขอบกล่องมาก
mAP@50-95	เฉลี่ยหลาย threshold ตั้งแต่ 0.5 ถึง 0.95	เข้มงวดกว่าและสะท้อน localization ดีขึ้น
IoU Threshold	เกณฑ์วัดพื้นที่ที่ซ้อนระหว่างกล่องทำนายกับ ground truth	ยิ่งสูง ยิ่งต้องทายกล่องแม่นยำ

ระวังในการจำ

False Positive = แจ้งเตือนผิด / Recall ต่ำ = ตรวจจับของจริงไม่ครบ / mAP@50-95 เข้มงวดกว่า mAP@50

Chapter 7: Implementation & Deployment

Section Implementation ในสไลด์สาธิตการเตรียม framework, init YOLO/RT-DETR, train, validate, export และรันบน Jetson Nano

7.1 เตรียม Framework และ Init Model

1. Install the framework

```
pip install ultralytics
```

2. Import libraries

```
from ultralytics import YOLO, RTDETR
```

```
import torch
```

โค้ดนี้ติดตั้ง ultralytics แล้ว import YOLO และ RTDETR สำหรับสร้างโมเดล detection ตามสไลด์

7.2 Train YOLO และ RT-DETR

Initialize YOLO (Nano version for Jetson Nano)

```
yolo_model = YOLO("yolov10n.pt")
```

```
yolo_results = yolo_model.train(
```

```
    data="your_custom_data.yaml",
```

```
    epochs=20,
```

```
    imgsz=640,
```

```
    batch=16,
```

```
    device=0,
```

```
    project="ITS_Workshop",
```

```
    name="YOLO_Run"
```

```
)
```

Initialize RT-DETR (ResNet18 backbone for speed)

```
rt detr_model = RTDETR("rt detr-resnet18.pt")
```

```
rt detr_results = rt detr_model.train(
```

```
    data="thai-traffic-micro/data.yaml",
```

```
    epochs=20,
```

```
    imgsz=640,
```

```
    batch=16,
```

```
    device=0,
```

```
    project="ITS_Workshop",
```

```
name="RTDETR_Run"  
)
```

พารามิเตอร์สำคัญคือ data เป็นไฟล์ dataset yml, epochs คือจำนวนรอบ train, imgsz คือขนาดภาพ, batch คือจำนวนตัวอย่างต่อรอบ และ device=0 คือใช้ GPU ตัวแรก

7.3 Validate และ Export

```
# Validate YOLO  
print("--- YOLO Metrics ---")  
yolo_val = yolo_model.val()  
  
# Validate RT-DETR  
print("--- RT-DETR Metrics ---")  
rtdetr_val = rtdetr_model.val()  
  
# Export YOLO to TensorRT with FP16  
yolo_model.export(  
    format="engine",  
    device=0,  
    half=True,  
    workspace=4  
)
```

TensorRT engine และ FP16 ช่วยให้รันบน edge device ได้เร็วขึ้น โดยสไลด์ยกตัวอย่างว่า FP16 quantization อาจช่วยเพิ่ม speed ได้ประมาณ 2x

7.4 Example Script บน Jetson Nano

```
from ultralytics import YOLO  
import cv2  
import time  
  
model = YOLO("yolov10n.engine", task="detect")  
cap = cv2.VideoCapture("test_traffic.mp4")  
  
while cap.isOpened():  
    success, frame = cap.read()
```

```

if not success:
    break
start_time = time.time()
results = model(frame, verbose=False)
latency_ms = (time.time() - start_time) * 1000
annotated_frame = results[0].plot()
cv2.putText(annotated_frame, f"Latency: {latency_ms:.1f} ms", (20, 50),
            cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)
cv2.imshow("Jetson Nano Edge Inference", annotated_frame)
if cv2.waitKey(1) & 0xFF == ord("q"):
    break
cap.release()
cv2.destroyAllWindows()

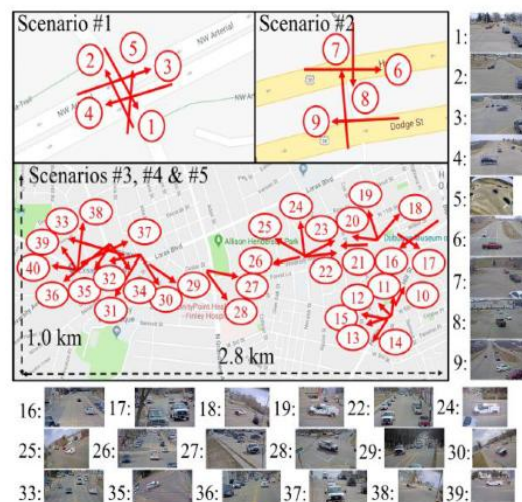
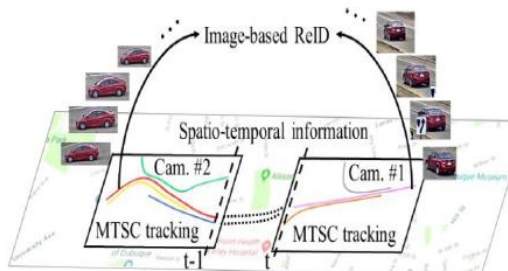
```

Chapter 8: Vehicle Re-Identification (Vehicle ReID)

Vehicle ReID คือการระบุวารถที่เห็นจากกล้องคนละตัวหรือคนละเวลา เป็นรถคันเดียวกันหรือไม่ จุดนี้สำคัญต่อ Smart City เพราะช่วย monitor, track และควบคุม traffic โดยใช้พลังของ CCTV

สไลด์ระบุความต้องการของ Smart Cities เช่น Safer City, Irresponsible Driving, Distracted Driving, Illegal Transportation และ Driving under Influence of Alcohol/Drugs ซึ่ง Vehicle ReID เปิดโอกาสให้ระบบติดตามเหตุการณ์ได้ต่อเนื่องมากขึ้น

Vehicle Re-Identification



รูปจากสไลด์หน้า 55: ภาพรวม Vehicle Re-Identification และการเชื่อมภาพรถข้ามกล้อง/ข้ามเส้นทาง

ตัวอย่างง่าย

รถคันเดียวกันถูกกล้อง A จับตอนเข้าเมือง และกล้อง B จับอีกครั้งอีกจุดหนึ่ง ระบบ ReID ต้องบอกให้ได้ว่าเป็นรถคันเดิม

8.1 ReID ต่างจาก Object Detection อย่างไร

งาน	คำถามหลัก	ผลลัพธ์
Object Detection	มีวัตถุอะไร และอยู่ตรงไหน	Bounding box + class
Vehicle ReID	รถภาพนี้เป็นคันเดียวกับรถในภาพอื่นหรือไม่	Similarity / ranking / identity matching

8.2 Normal Pipeline ใน Vehicle ReID

Normal Pipeline in Vehicle ReID

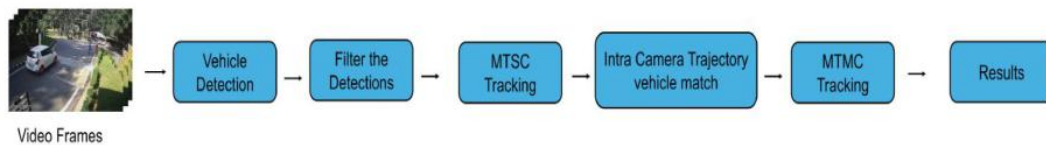


FIGURE 5. The vehicle tracking pipeline starts with vehicle detection, followed by filtering to remove incorrect detections. Within each camera, vehicle tracklets are created using single-camera tracking. These tracklets are then matched with tracklets from neighboring cameras for multi-camera, multi-target tracking.

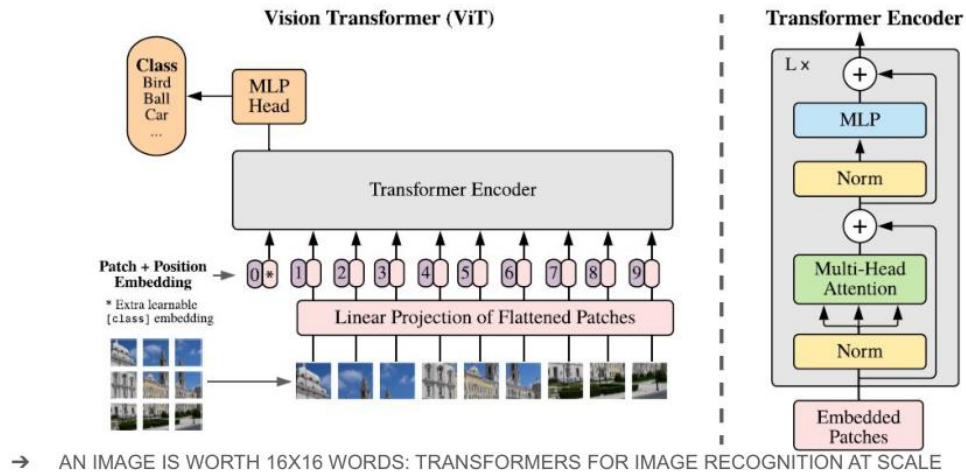
รูปจากสไลด์หน้า 65: Pipeline เริ่มจาก video frames -> vehicle detection -> filter detections -> MTSC tracking -> inter-camera matching -> MTMC tracking -> result

- Vehicle Detection: ตรวจจับรถในแต่ละ frame
- Filter the Detections: กรองผลตรวจจับที่ผิด
- MTSC Tracking: ติดตามรถในกล้องเดียว
- Inter-Camera Tracking vehicle match: จับคู่รถระหว่างกล้องใกล้เคียง
- MTMC Tracking: ติดตามหลายกล้องหลายเป้าหมาย
- Result: ได้ trajectory หรือประวัติการเดินทางของรถ

Chapter 9: Transformer Architecture for ReID

สไลด์เชื่อมจาก Transformer ดั้งเดิมไปสู่ Vision Transformer (ViT) และ TransReID โดยใจความคือ Transformer ใช้ self-attention เพื่อมองความสัมพันธ์ของส่วนต่าง ๆ ในภาพ และช่วยโฟกัสส่วนที่จำแนกตัวตนได้ดี

Transformer Revolution



รูปจากสไลด์หน้า 58: Vision Transformer แปลงภาพเป็น patches เหมือนคำในประโยค

9.1 Patch และ Self-Attention

Vision Transformer มองภาพเป็นชุดของ patch เช่น 16x16 pixels แล้วแปลง patch เหล่านั้นเป็น token คล้ายกับคำในประโยค จากนั้น self-attention จะช่วยให้เราทราบว่า patch ใดสำคัญและสัมพันธ์กับ patch อื่นอย่างไร

Visualization of Attention in Transformer-based Method

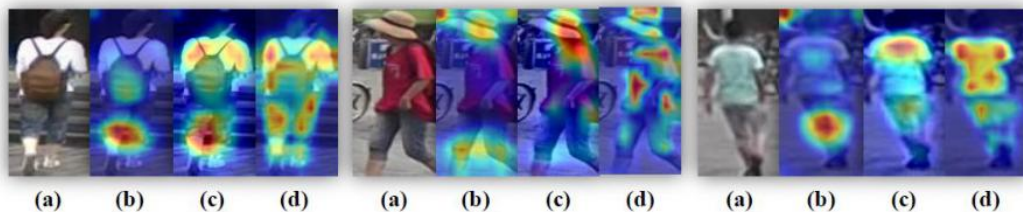


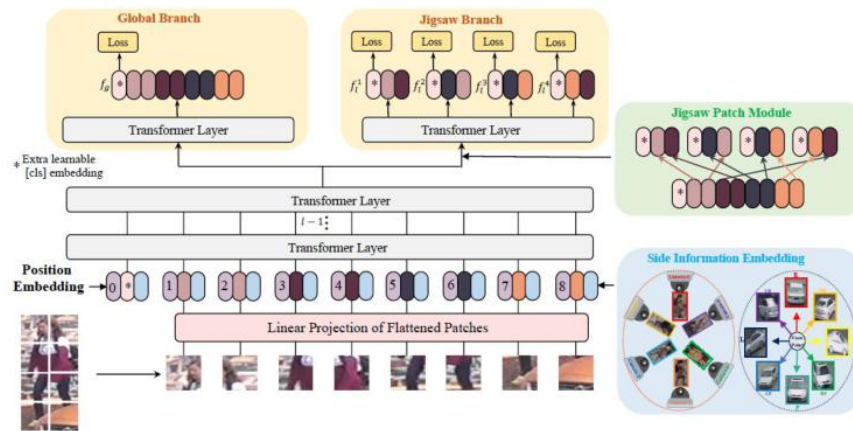
Figure 1: Grad-CAM [34] visualization of attention maps: (a) Original images, (b) CNN-based methods, (c) CNN+attention methods, (d) Transformer-based methods which captures global context information and more discriminative parts.

รูปจากสไลด์หน้า 59: Attention map แสดงว่า transformer-based method จับ global context และ discriminative parts ได้ดี

9.2 TransReID

TransReID คือ Transformer-based Object Re-Identification ใช้ transformer layer, patch embedding, global branch และ local/jigsaw branch เพื่อเรียนรู้ feature สำหรับการระบุตัวตนของ object

TransReID



→ TransReID: Transformer-based Object Re-Identification

รูปจากสไลด์หน้า 60: TransReID architecture สำหรับ Object Re-Identification

Attention Map in TransReID

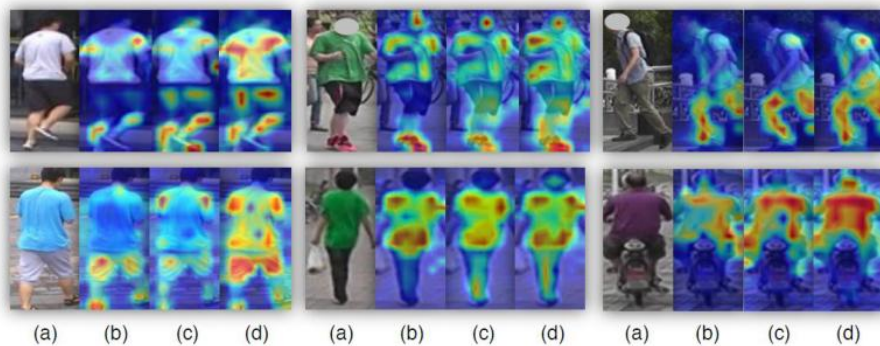


Figure 5: Grad-CAM visualization of attention maps. (a) Input images, (b) Baseline, (c) JPM w/o rearrange, (d) JPM.

→ TransReID: Transformer-based Object Re-Identification

รูปจากสไลด์หน้า 61: Attention Map ใน TransReID เปรียบเทียบ baseline และ JPM

Chapter 10: Vehicle ReID Datasets

สไลด์รวบรวม dataset สำหรับ Vehicle ReID หลายชุด เช่น COMPCARS, VehicleID, VeRi-776, VehicleX, MCU-VREID, CityFlow, RGBN300, VRAI และ UAV-VeID โดยในเอกสารนี้สรุปชุดที่มีรายละเอียดตัวเลขในสไลด์

VeRi-776 Datasets



Fig. 2. Vehicle samples selected from the VeRi dataset. All images are normalized to the same size. (a) Diversity of vehicle colors and types. (b) Variation of the viewpoints, illuminations, resolutions, and occlusions for the vehicles in different cameras.

รูปจากสไลด์หน้า 68: ตัวอย่างภาพจาก VeRi-776 แสดงสี ประเภทรถ มุมมอง แสง ความละเอียด และ occlusion ที่แตกต่างกัน

Dataset	จำนวนกล้อง/Platform	จำนวนภาพ/วิดีโอ	จำนวนรถ	จุดเด่น/ใช้กับงานแบบไหน
VeRi-776	20 CCTVs	51,035 images	776 identical vehicles, 218 viewpoints	Urban surveillance และ ReID หลายมุมมอง
MCU-VReID	42 CCTVs + UAVs	5,630 images	51 identical vehicles	Cross Platform ระหว่าง CCTV และ UAV
CityFlow	40 cameras จาก 10 intersections ใน US city	>200,000 annotated BBox, videos 960p + 10FPS	666 identical vehicles	MTMC vehicle tracking ระดับเมือง
RGBN300	8 camera views + multi-spectral	50,125 RGB + 50,125 NIR + 17,250 thermal IR	300 identical vehicles; thermal 100 vehicles	งาน multi-spectral RGB/NIR/Thermal
VRAI	Two UAVs	350 video clips	13,022 identical vehicles	ต่าง location, flight altitude และ angles

VeRi-776 Dataset

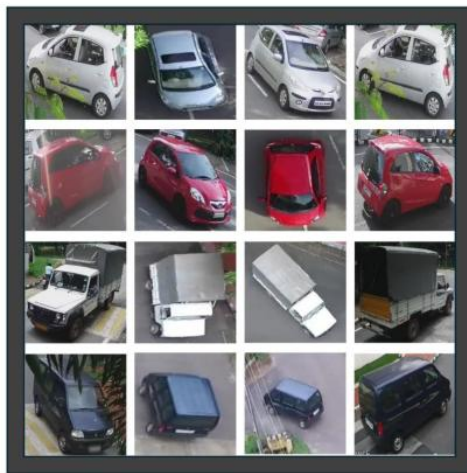


- 20 CCTVs
- 51,035 images
- 776 identical vehicles
- 218 viewpoints

FIGURE 6. Sample vehicle images from VeRi-776 dataset.

รูปจากสไลด์หน้า 69: รายละเอียด VeRi-776: 20 CCTVs, 51,035 images, 776 vehicles, 218 viewpoints

MCU-VReID



- 42 CCTVs + UAVs
- Use for Cross Platform
- 51 Identical Vehicles
- 5,630 Images

FIGURE 7. Sample images from MCU-VReID dataset.

รูปจากสไลด์หน้า 70: MCU-VReID ใช้กับ Cross Platform ระหว่าง CCTV และ UAV

CityFlow Dataset

- 40 Cameras from 10 Intersection in US City
- Largest Dataset
- > 200,000 annotated BBox
- 666 identical vehicles
- Videos 960p + 10FPS
- Designed for research in MTMC vehicle tracking



FIGURE 8. Sample Images form CityFlow dataset.

รูปจากสไลด์หน้า 71: CityFlow สำหรับ MTMC vehicle tracking ระดับเมือง

RGBN300 Dataset



FIGURE 9. Sample images from RGBN300 dataset.

- Multi-Spectral Images
- 300 Identical Vehicles
- 8 Camera Views
- 50,125 RGB images
- 50,125 NIR (Near InfraRed) images
- Additional 17,250 Thermal Infrared Images of 100 Vehicles

รูปจากสไลด์หน้า 72: RGBN300 รวม RGB, NIR และ Thermal Infrared

VRAI Dataset

- 350 video clips
- 13,022 identical vehicles
- Two UAVs
- Different Location, Flight Altitude and Angles



FIGURE 10. Sample images from VRAI dataset.

รูปจากสไลด์หน้า 73: VRAI dataset จาก UAV หลายมุมและความสูง

Chapter 11: Fast ReID Framework

Fast ReID Framework สามารถเข้าใจเป็น pipeline จากภาพรถเข้าระบบ -> เตรียมภาพ -> backbone ดึง feature -> aggregation รวม feature -> head ปรับ vector -> loss function ช่วย train -> distance metric เปรียบเทียบ query กับ gallery

Pipeline of Fast ReID

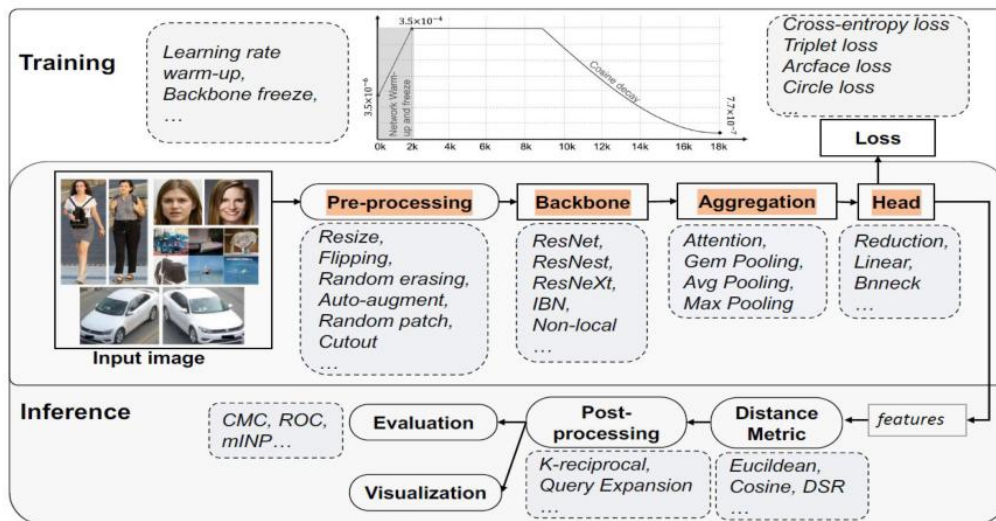


Figure 1. The Pipeline of FastReID library.

รูปจากสไลด์หน้า 76: Pipeline ของ Fast ReID

11.1 Query, Gallery และ Ranking

- Query Image: ภาพรถที่ต้องการค้นหา

- Gallery Image: ชุดภาพรถที่ระบบใช้เปรียบเทียบ
- Feature Extraction: แปลงภาพเป็น feature vector
- Similarity Measurement: วัดความคล้ายระหว่าง vector
- Ranking Result: เรียงภาพที่คล้ายที่สุดขึ้นมาก่อน

ภาพใหญ่ของ ReID

ReID ไม่ได้จำรูปแบบมนุษย์ด้วยตา แต่แปลงภาพเป็น vector แล้ววัดว่าภาพไหนอยู่ใกล้กันที่สุดใน feature space

Chapter 12: Pre-processing และ Augmentation

Pre-processing และ augmentation ช่วยให้โมเดล robust ต่อ noise และสภาพแวดล้อมจริง เช่น มุมกล้องต่าง แสงต่าง และรถถูกบัง

Image Pre-processing & Augmentation

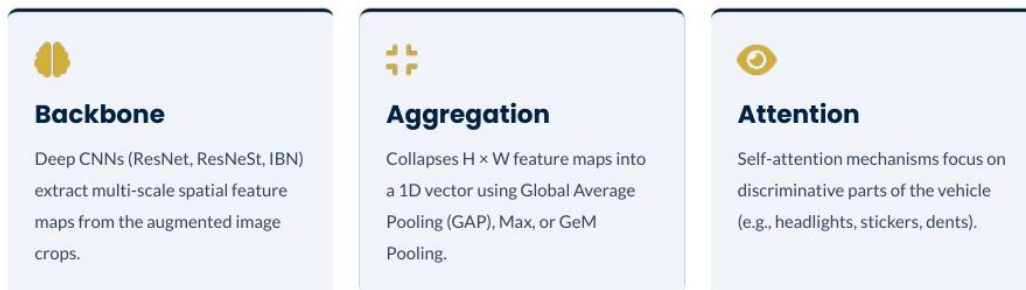
To make the model robust against environmental noise, we apply aggressive data augmentation:

- 📐 **Resize & Padding:** Normalizing input dimensions.
- ↔ **Random Flip:** Simulating different camera angles.
- 🚧 **Random Erasing:** Simulating occlusions (e.g., other cars).
- 🔧 **Auto-Augment:** Learning optimal augmentation strategies.

รูปจากสไลด์หน้า 78: Image Pre-processing & Augmentation เพื่อเพิ่ม robustness ต่อ environmental noise

เทคนิค	ความหมาย	ช่วยอะไร
Resize	ปรับขนาดภาพให้เข้ารูปแบบเดียวกัน	ทำให้ input เข้าโมเดลได้สม่ำเสมอ
Padding	เติมขอบภาพเพื่อรักษาสัดส่วน	ลดการบิดเบี้ยวของรถ
Random Flip	สุ่มกลับภาพซ้าย-ขวา	จำลองมุมกล้องที่ต่างกัน
Random Erasing	สุ่มลบบางส่วนของภาพ	จำลอง occlusion เช่น รถถูกบัง
Auto-Augment	เรียนรู้กลยุทธ์ augmentation ที่เหมาะสม	เพิ่มความ robust อัตโนมัติ
Random Patch	สุ่ม patch เพื่อเปลี่ยนบางส่วนของภาพ	ช่วยให้โมเดลไม่จำเฉพาะ pattern เดิม

Feature Extraction & Aggregation



รูปจากสไลด์หน้า 80: Feature Extraction & Aggregation: backbone, aggregation และ attention

13.1 Backbone

Backbone คือส่วนที่ดึง feature จากภาพ เช่น ResNet, ResNeSt, ResNeXt, IBN และ Non-Local ตามที่สไลด์แสดง โมเดลจะเปลี่ยนภาพรถเป็น feature maps ที่มีข้อมูลเชิงพื้นที่หลายระดับ

13.2 Aggregation

Aggregation คือการบีบข้อมูลภาพขนาดใหญ่จาก feature map รูปแบบ $C \times H \times W$ ให้กลายเป็น vector 1 มิติที่ใช้แทนภาพทั้งภาพ

Aggregation

- Collapse 3D Tensor ($C \times H \times W$) → 1D vector ($C \times 1$)
- Global Avg Pooling
 - ◆ Average all pixels in each channel
 - ◆ ReID : GAP → Global Summary of the image
- Max Pooling
 - ◆ Max all pixels in each channel
 - ◆ ReID : MAP → Capture High Outstanding Features (Bright glow of distinct vehicle parts)
- GeM (Generalized Mean) Pooling
 - ◆ Balance between GAP and MAP
 - ◆ Dynamic Finding optimal parameter p that define how much details the model need between GAP and MAP
- Attention Pooling
 - ◆ Avoid blindly treat all pixels equally : Attention Map $H \times W$
 - ◆ Masked pixels by Attention before GAP / MAP

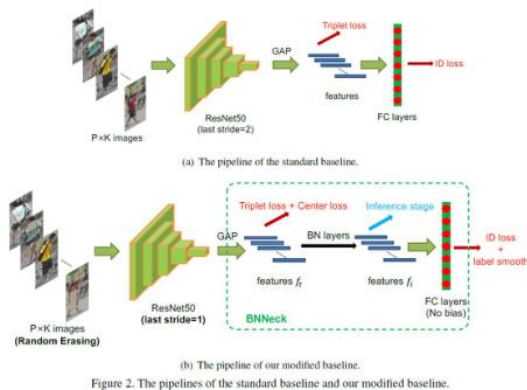
รูปจากสไลด์หน้า 82: Aggregation เปลี่ยน 3D Tensor เป็น 1D vector และเปรียบเทียบ GAP, Max, GeM, Attention Pooling

วิธี Aggregation	แนวคิด	เหมาะกับอะไร
Global Average Pooling (GAP)	เฉลี่ยทุก pixel ในแต่ละ channel	สรุปภาพแบบ global summary
Max Pooling	เลือกค่ามากสุดในแต่ละ channel	จับ feature เด่นมาก เช่น จุดสว่างหรือส่วนเฉพาะของรถ
GeM Pooling	สมดุลระหว่าง GAP และ Max ผ่าน parameter p	หาความละเอียดที่เหมาะสมแบบ dynamic
Attention Pooling	ใช้ attention map เลือก pixel สำคัญก่อน pooling	ไม่ treat ทุก pixel เท่ากัน เหมาะกับรายละเอียดจำแนกตัวตน

13.3 Head และ BNNeck

Head เป็นส่วนท้ายของโมเดลที่ทำให้ feature เหมาะกับการคำนวณ loss และการเปรียบเทียบ โดยสไลด์ระบุ Reduction Layer, Linear Layer และ BNNeck Layer

The Re-ID Head: BNNeck Design



BNNeck Innovation

Standard Re-ID heads often struggle to balance Classification Loss and Triplet Loss simultaneously. BNNeck solves this by:

- Separating feature spaces for different losses.
- **Before BN:** Feature used for Triplet Loss (distance).
- **After BN:** Feature used for Cross-Entropy (ID).

รูปจากสไลด์หน้า 86: BNNeck Design แยก feature space สำหรับ Triplet Loss และ Cross-Entropy

BNNeck แก้ปัญหาความขัดแย้งระหว่าง Triplet Loss และ Cross Entropy Loss โดยใช้ feature ก่อน BN สำหรับ Triplet Loss และ feature หลัง BN สำหรับ Cross-Entropy Loss

13.4 Loss Functions

Loss	แนวคิด	ใช้ใน ReID เพื่ออะไร
Cross-Entropy Loss	จำแนกแต่ละ object เป็น ID class	ใช้เป็น ID loss
Triplet Loss	ดึง Anchor กับ Positive ให้ใกล้ และผลัก Negative ให้ออกห่าง	เรียนรู้ระยะใน Euclidean space
ArcFace Loss	บังคับ embedding บน hypersphere และใช้มุม	ทำ cluster ของแต่ละ vehicle ID ให้แน่น
Circle Loss	ถ่วงน้ำหนัก positive/negative pair แบบ dynamic	ลดการเสียแรงคำนวณกับ pair ที่ดีพอแล้ว

Triplet Loss

- Concept of “Classes”
- Used for optimizing Euclidean Distance between Feature vectors
- Look for 3 images at the same time :
 - ◆ Anchor (a) : Baseline image for Car X
 - ◆ Positive (b) : Different image for Car X
 - ◆ Negative (n) : Image of Car Y
- $L_{\text{triplet}} = \max(0, (\|f(a) - f(p)\|_2)^2 - (\|f(a) - f(n)\|_2)^2 + \alpha)$
- Distance between Anchor & Positive => should be small.
- Distance between Anchor & Negative => should be large
- $L_{\text{triplet}} \rightarrow$ Force the model to learn to ensure that negative images are farther away than positive images at least margin “alpha”

รูปจากสไลด์หน้า 92: Triplet Loss ใช้ Anchor, Positive และ Negative พร้อม margin alpha

13.5 Distance Matrix และ Similarity Search

หลังได้ feature vector แล้ว ระบบต้องเปรียบเทียบ query กับ gallery ทุกภาพ Distance Matrix คือ matrix ขนาด $M \times N$ ที่ $D(i,j)$ แทนระยะระหว่าง query ที่ i กับ gallery ที่ j

Distance Matrix

- Pairwise comparison between every query and every gallery
- Definition :
 - ◆ M : number of query images
 - ◆ N : number of gallery images
 - ◆ d : dimension of feature embeddings (e.g., $d = 2048$)
 - ◆ Query Matrix (Q) : $M \times d$ (each row = query vector, q)
 - ◆ Gallery Matrix (G) : $N \times d$ (each row = gallery vector, g)
- Distance Matrix :
 - ◆ $M \times N$ matrix where $D(i,j)$ denote to the distance between i -th query and j -th gallery

รูปจากสไลด์หน้า 95: Distance Matrix: pairwise comparison ระหว่าง query matrix และ gallery matrix

Similarity Search: Distance Metrics

Metric	Computational Cost	Use Case
Euclidean Distance	Low (O(N))	Standard spatial similarity.
Cosine Similarity	Medium	Directional feature alignment.
Jaccard Distance	High	Re-ranking with neighbor sets.

Cosine Similarity:

$$\text{Similarity} = \frac{A \cdot B}{|A| |B|}$$

รูปจากสไลด์หน้า 96: Distance Metrics เช่น Euclidean, Cosine และ Jaccard

Metric	ความหมาย	ข้อควรจำ
Euclidean Distance (L2)	ระยะเส้นตรงใน N-dimension	ไวต่อ magnitude เช่น แสง เงา contrast ต้อง normalize
Cosine Similarity/Distance	วัดมุมระหว่าง vector	similarity ใกล้ 1 แปลว่าคล้ายมาก
Jaccard Distance	ใช้กับ re-ranking และ neighbor sets	ต้นทุนคำนวณสูงกว่า

13.6 Evaluation สำหรับ ReID

Metrics for Evaluation

- CMC Rank-1, 5, 10
- ROC
- mAP

- CMC Rank-1: ภาพอันดับ 1 ตรงกับตัวจริงหรือไม่
- CMC Rank-5/10: ตัวจริงอยู่ใน 5 หรือ 10 อันดับแรกหรือไม่
- mAP: วัดคุณภาพ ranking โดยรวม
- ROC: วิเคราะห์ trade-off ของการแยกคู่ที่ใช้/ไม่ใช่

Chapter 14: City Scale ReID และ Spatial-Temporal Integration

ช่วงท้ายสไลด์ขยายจาก ReID ระดับภาพเดี่ยวไปสู่ City Scale Multi-Camera Vehicle Tracking โดยใช้ crossroad zones, multi-camera tracklet matching และข้อมูลเชิงพื้นที่/เวลา

MTMC Pipeline

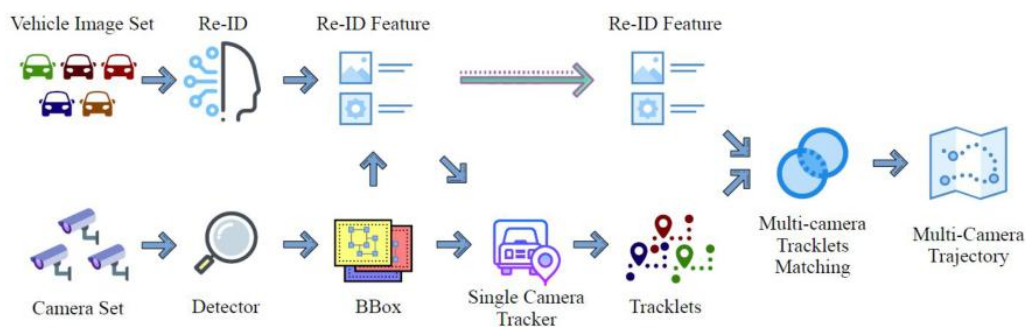


Figure 2. **Pipeline of Multi-Target Multi-Camera.** The MTMCT system first uses the detector to obtain the BBox of the target from each frame of each camera video; then uses the trained Re-ID model to extract the apparent features of the target BBox; the single-camera tracker uses BBox and Re-ID feature for each tracking target to generate single-camera tracklets; finally, according to the single-camera tracklets and Re-ID feature, the ID synchronization between cameras generates a multi-camera trajectory.

Crossroad Zones

- Zoning to determine the vehicle directions at each intersection

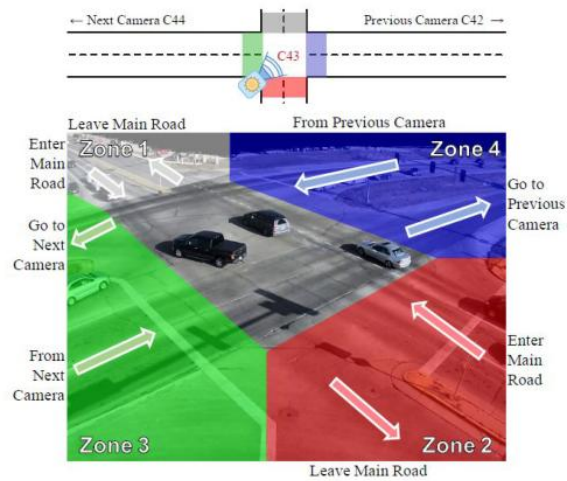


Figure 4. **Crossroad zones for C43.** According to the characteristics of the crossroad, we divide the video frame into four zones to divide the vehicle tracklets.

รูปจากสไลด์หน้า 106: Crossroad Zones ใช้กำหนดทิศทางการที่แต่ละ intersection

Multi-Camera Tracklets Matching

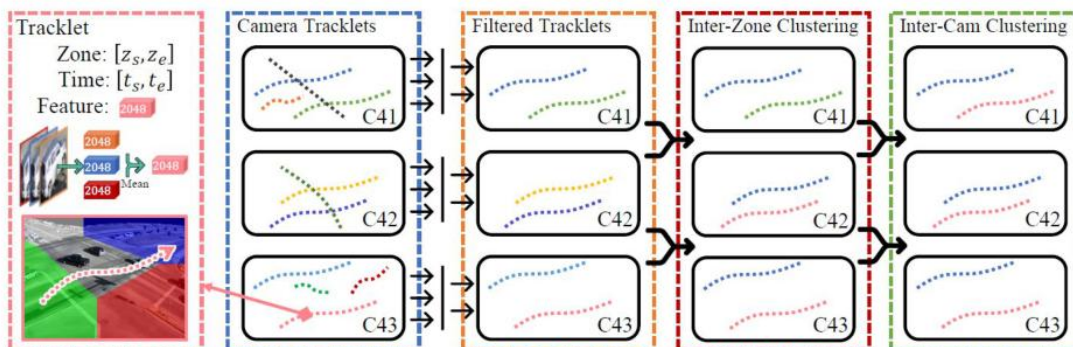


Figure 5. **Process of Multi-camera Tracklets Matching.** 1) Generate the necessary information for matching according to crossroad zones and tracklets; 2) Use TFS to filter the tracklets; 3) Calculate the similarity matrix between the tracklets and use DBTM to perform matching constraints; 4) Perform SCAC process between tracklets, perform inter-zone clustering and inter-cam clustering respectively.

รูปจากสไลด์หน้า 107: Multi-Camera Tracklets Matching สำหรับจับคู่ tracklet ระหว่างกล้อง

Proposed Pipeline

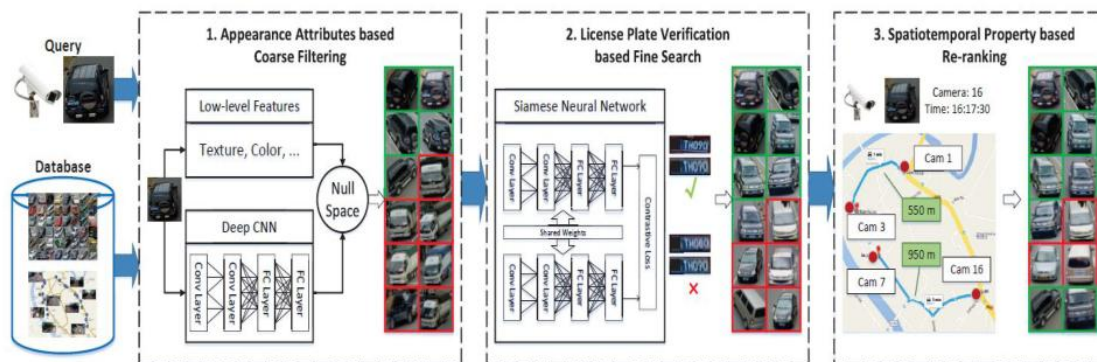


Fig. 3: The architecture of the PROVID framework.

รูปจากสไลด์หน้า 112: Proposed Pipeline ในตัวอย่าง City Scale ReID

Spatial and Temporal Data Integration

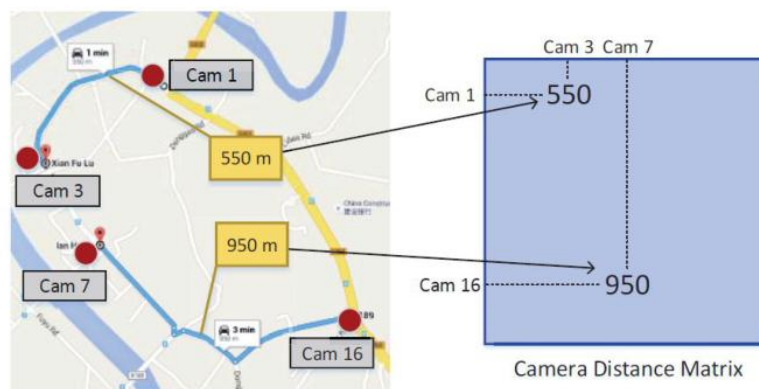


Fig. 7: The physical distance matrix of surveillance cameras.

รูปจากสไลด์หน้า 114: Spatial and Temporal Data Integration เชื่อมพื้นที่และเวลาในการวิเคราะห์

ใจความคือ ReID ระดับเมืองไม่ได้ดูแค่ภาพว่าคล้ายกันหรือไม่ แต่ต้องเอาเวลา ตำแหน่งกล้อง ทิศทางถนน และความเป็นไปได้ของเส้นทางเข้ามาช่วย เช่น รถไม่ควรโผล่จากกล้องหนึ่งไปอีกกล้องหนึ่งในเวลาที่ไปไม่ได้

จำลอง

City Scale ReID = Visual similarity + Spatial constraint + Temporal constraint + Tracklet matching

สรุปภาพรวมทั้งหมด

- AI Vision เปลี่ยน ITS จากระบบ sensor-based แบบเดิมให้มองเห็นและตีความวัตถุบนถนนได้ละเอียดขึ้น
- Camera กลายเป็น sensor สำคัญ เพราะกล้องหนึ่งตัวทำได้ทั้ง detection, recognition และ identification
- Edge Computing ลด bandwidth จากวิดีโอระดับ Mbps ให้เหลือ metadata ขนาดเล็ก และลด latency ใกล้เคียงแหล่งข้อมูล
- Object Detection ตอบคำถามว่า “อะไร” และ “อยู่ตรงไหน”
- YOLO เหมาะกับ real-time เพราะเป็น one-stage detector ที่เร็วมาก
- RT-DETR ใช้ Transformer มอง global context จึงเหมาะกับฉากซับซ้อนและ occlusion สูงกว่าในบางสถานการณ์
- Evaluation Metrics เช่น Precision, Recall, AP, mAP และ IoU ใช้วัดทั้งความถูกต้องและคุณภาพของ bounding box
- Vehicle ReID ช่วยติดตามรถคันเดิมข้ามกล้อง เป็นหัวใจของ Smart City surveillance และ MTMC tracking
- Transformer และ attention ช่วยให้ ReID จับส่วนสำคัญของรถได้ดีขึ้น
- City Scale ReID ต้องรวม visual feature กับ spatial-temporal information เพื่อให้การจับคู่สมเหตุสมผล

กล่องจำง่าย

Object Detection = หา “อะไร” และ “อยู่ตรงไหน”

YOLO = เร็ว เหมาะกับ Real-time

RT-DETR = เข้าใจภาพรวมดี เหมาะกับฉากซับซ้อน

Edge Computing = ประมวลผลใกล้กล้อง ลด Bandwidth

Vehicle ReID = จำรถคันเดิมข้ามกล้อง

BNNeck = แยก feature สำหรับ Triplet Loss และ CE Loss

Distance Matrix = ตารางเปรียบเทียบ query ทุกภาพกับ gallery ทุกภาพ

Glossary: คำศัพท์สำคัญ

คำศัพท์	คำอธิบายภาษาไทย
ITS	Intelligent Transport System ระบบขนส่งอัจฉริยะ
Edge Computing	การประมวลผลใกล้แหล่งข้อมูล เช่น ใกล้กล้อง CCTV
Object Detection	งานตรวจจับวัตถุว่าอยู่ตรงไหนและเป็นอะไร
Bounding Box	กรอบสี่เหลี่ยมที่ล้อมวัตถุในภาพ
Classification	การจำแนกคลาสของวัตถุ
YOLO	You Only Look Once โมเดล object detection แบบ one-stage ที่เร็ว

RT-DETR	Real-Time Detection Transformer โมเดล detection ที่ใช้ Transformer
CNN	Convolutional Neural Network เก่งการดึง local features
Transformer	สถาปัตยกรรมที่ใช้ self-attention เพื่อจับความสัมพันธ์ทั้งภาพ
Self-Attention	กลไกที่ให้โมเดลเลือกส่วนสำคัญและความสัมพันธ์ของข้อมูล
NMS	Non-Max Suppression วิธีลบ bounding boxes ซ้ำซ้อน
IoU	Intersection over Union อัตราส่วนพื้นที่ซ้อนของกล่องทำนายและ ground truth
mAP	mean Average Precision ค่าเฉลี่ย AP หลายคลาส
ReID	Re-identification การระบุว่าภาพต่าง ๆ เป็น object/vehicle เดียวกันหรือไม่
Feature Vector	เวกเตอร์ตัวแทนภาพที่ใช้เปรียบเทียบความคล้าย
Similarity Measurement	การวัดความคล้ายกันระหว่าง feature vectors
BNNeck	Batch Normalization Neck ที่แยก feature space สำหรับ loss ต่างชนิด
Triplet Loss	Loss ที่ใช้ Anchor, Positive, Negative เพื่อเรียนรู้ระยะห่าง
CMC Rank	Metric ของ ReID ที่ดูว่าคำตอบอยู่ในอันดับที่เท่าไร
MTMC Tracking	Multi-Target Multi-Camera Tracking การติดตามหลายวัตถุหลายกล้อง

คำถามทบทวนท้ายบท 20 ข้อ พร้อมเฉลย

19. 1. ITS แบบเดิมมี bottleneck หลักอะไร

เฉลย: พึ่งพา sensor-based devices จำนวนมาก ต้นทุนสูง ภาพไม่ใช่ข้อมูลหลัก และมองรายละเอียดวัตถุได้จำกัด

20. 2. ทำไมกล้องจึงกลายเป็น sensor สำคัญใน ITS

เฉลย: เพราะกล้องราคาถูกลง เข้าถึงง่าย และภาพเดียวใช้ทำ detection, recognition, identification ได้หลายงาน

21. 3. ทำไมการส่งวิดีโอ 4K จากทุกกล้องขึ้น cloud ถึงไม่เหมาะ

เฉลย: ใช้ bandwidth สูง เช่น 8 Mbps ต่อกล้อง และเมื่อขยายเป็น 100/1000 กล้องจะมีภาระ uplink สูงมาก

22. 4. Edge Computing ลดปัญหาอะไร

เฉลย: ลด bandwidth, latency และเพิ่ม resilience โดยส่งเฉพาะ metadata/ภาพจำเป็นแทนวิดีโอเต็ม

23. 5. Object Detection ต้องตอบคำถามอะไรบ้าง

เฉลย: วัตถุอยู่ตรงไหน และวัตถุนั้นคืออะไร

24. 6. Two-stage detector ต่างจาก one-stage detector อย่างไร

เฉลย: Two-stage หา region ก่อนแล้ว classify ส่วน one-stage ทำนายกล่องและคลาสพร้อมกัน

25. 7. YOLO ย่อมาจากอะไรและเด่นเรื่องใด

เฉลย: You Only Look Once เด่นเรื่องความเร็วและ real-time detection

26. 8. Grid ใน YOLO มีหน้าที่อะไร

เฉลย: แบ่งภาพให้แต่ละ cell ช่วยทำนาย bounding box, confidence และ class probability

27. 9. IoU ใช้ทำอะไร

เฉลย: วัดพื้นที่ซ้อนระหว่างกล่องทำนายกับ ground truth เพื่อประเมินว่าทายถูกไหม

28. 10. NMS คืออะไรและทำไมเป็น bottleneck

เฉลย: วิธีคัดกล่องซ้ำซ้อนออก แต่เมื่อวัตถุซ้อนกันมาก candidate boxes เยอะทำให้ latency สูง

29. 11. RT-DETR คืออะไร

เฉลย: Real-Time Detection Transformer เป็นโมเดล detection ที่ใช้ Transformer และเป็น NMS-Free

30. 12. CNN และ Transformer ต่างกันอย่างไรในบริบท ITS

เฉลย: CNN เก่ง local feature ส่วน Transformer เก่ง global context และฉาก occlusion

31. 13. Precision ต่ำแปลว่าอะไร

เฉลย: โมเดลมี false alarm หรือ false positive เยอะ

32. 14. Recall ต่ำแปลว่าอะไร

เฉลย: โมเดลตรวจจับวัตถุจริงไม่ครบหรือ miss target เยอะ

33. 15. mAP@50-95 เข้มงวดกว่า mAP@50 อย่างไร

เฉลย: เพราะเฉลี่ยหลาย IoU threshold ตั้งแต่ 0.5 ถึง 0.95 ต้องทายกรอบแม่นยำกว่า

34. 16. Vehicle ReID ต่างจาก Object Detection อย่างไร

เฉลย: Detection หาวัตถุในภาพ ส่วน ReID จับวารถจากหลายภาพ/หลายกล้องเป็นคันเดียวกันไหม

35. 17. Pipeline ReID ปกติเริ่มจากอะไรไปอะไร

เฉลย: video frame -> detection -> filter -> single-camera tracking -> inter-camera matching -> multi-camera tracking
-> result

36. 18. Vision Transformer มองภาพอย่างไร

เฉลย: แบ่งภาพเป็น patches แล้วใช้ self-attention เรียนรู้ความสัมพันธ์ของ patch ต่าง ๆ

37. 19. BNNeck ช่วยแก้ปัญหาอะไร

เฉลย: ช่วยแยก feature space สำหรับ Triplet Loss และ Cross-Entropy เพื่อลดความขัดแย้งในการ train

38. 20. City Scale ReID ต้องใช้อะไรนอกจาก visual similarity

เฉลย: ต้องใช้ spatial constraint, temporal constraint, direction/zone และ tracklet matching

แนวข้อสอบเลือกตอบ 20 ข้อ พร้อมเฉลย

39. 1. ข้อใดคือข้อจำกัดของ sensor-based ITS แบบเดิม

- A. ติดตั้งง่ายมาก
- B. มองรายละเอียดวัตถุได้ลึกมาก
- C. ต้องใช้ sensor จำนวนมากและต้นทุนสูง
- D. ไม่ต้องใช้การสื่อสาร

เฉลย: C

40. 2. กล้อง 1 ตัวใน Vision-AI สามารถทำงานใดได้

- A. Object Detection
- B. Object Recognition
- C. Object Identification
- D. ถูกทุกข้อ

เฉลย: D

41. 3. Bitrate ที่สไลด์ยกตัวอย่างสำหรับ 4K 30FPS H.265 คือประมาณเท่าไร

- A. 800 Mbps
- B. 8 Mbps
- C. 2 KB
- D. 30 Gbps

เฉลย: B

42. 4. ข้อใดเป็นผลลัพธ์ที่ edge node ส่งขึ้น cloud ตามแนวคิดในสไลด์

- A. raw video ทั้งหมด
- B. JSON metadata และภาพเล็ก
- C. เฉพาะเสียง
- D. ไม่มีข้อมูลใดส่งขึ้น cloud

เฉลย: B

43. 5. Object Detection ต้องทำสองงานหลักคืออะไร

- A. Resize และ padding
- B. Classification และ clustering
- C. Bounding box และ classification
- D. Training และ export

เฉลย: C

44. 6. Fast R-CNN / Faster R-CNN จัดเป็น detector ประเภทใด

- A. One-stage
- B. Two-stage
- C. NMS-Free เท่านั้น
- D. ReID framework

เฉลย: B

45. 7. YOLO จัดเป็น detector ประเภทใด

- A. One-stage
- B. Two-stage
- C. Dataset
- D. Loss function

เฉลย: A

46. 8. YOLO ได้ชื่อว่า You Only Look Once เพราะอะไร

- A. ใช้ภาพเดียวทำนายกล่องและคลาสพร้อมกัน
- B. ใช้กล่องเดียวเสมอ
- C. train ได้ครั้งเดียว
- D. ไม่ต้องใช้ GPU

เฉลย: A

47. 9. NMS ใช้เพื่ออะไร

- A. เพิ่มจำนวนกล่องซ้ำ
- B. ลด bounding boxes ซ้ำซ้อน
- C. ทำ data augmentation
- D. แปลงภาพเป็น patch

เฉลย: B

48. 10. สถานการณ์ใดทำให้ NMS latency สูง

- A. ถนนว่าง
- B. วัตถุซ้อนกันจำนวนมาก
- C. ภาพไม่มีรถ
- D. ไม่มี candidate boxes

เฉลย: B

49. 11. RT-DETR ใช้แนวคิดหลักใด

- A. Transformer
- B. RAID
- C. SQL Join
- D. Normalization

เฉลย: A

50. 12. YOLO เหมาะกับ use case ใดตามตาราง

- A. Complex Urban Intersections เท่านั้น
- B. Highways, Toll Booths, Parking Lots
- C. งาน OCR เท่านั้น
- D. งาน Database

เฉลย: B

51. 13. RT-DETR เติ่นเรื่องใด

- A. Severe occlusion และ global context
- B. ไม่ต้องใช้ข้อมูล
- C. ไม่มี latency
- D. ไม่ต้องใช้ hardware

เฉลย: A

52. 14. Precision = ?

- A. TP / Total Predictions
- B. TP / Total Ground Truths
- C. FP / TP
- D. IoU / AP

เฉลย: A

53. 15. Recall = ?

- A. TP / Total Ground Truths

- B. TP / Total Predictions
- C. AP เฉลี่ย
- D. FPS เฉลี่ย

เฉลย: A

54. 16. Vehicle ReID คืออะไร

- A. ตรวจจับวาร์อยู่ตรงไหน
- B. ระบวาร์รถจากหลายภาพเป็นคันเดียวกันหรือไม่
- C. ลดขนาดภาพ
- D. export model

เฉลย: B

55. 17. VeRi-776 มีจำนวน identical vehicles เท่าไรตามสไลด์

- A. 51
- B. 666
- C. 776
- D. 13,022

เฉลย: C

56. 18. CityFlow ออกแบบสำหรับงานใด

- A. MTMC vehicle tracking
- B. เฉพาะ face recognition
- C. เฉพาะ sentiment analysis
- D. เฉพาะ audio

เฉลย: A

57. 19. Triplet Loss ใช้ภาพก๊อปปี้หลัก

- A. Anchor, Positive, Negative
- B. Query, Key, Value เท่านั้น

- C. Only Positive
- D. Only Negative

เฉลย: A

58. 20. City Scale ReID ควรใช้ข้อมูลใดร่วมกับ visual feature

- A. Spatial และ Temporal data
- B. เฉพาะสีรถเท่านั้น
- C. เฉพาะ FPS
- D. เฉพาะ batch size

เฉลย: A

แนวข้อสอบเขียน 10 ข้อ พร้อมแนวคำตอบ

59. 1. อธิบายว่าทำไม Edge Computing จึงสำคัญต่อ AI Vision ใน ITS

แนวคำตอบ: ควรกล่าวถึง bandwidth ของวิดีโอ 4K, latency, การส่งเฉพาะ JSON/image ขนาดเล็ก และความสามารถในการทำ inference ใกล้กล้อง

60. 2. เปรียบเทียบ Edge Computing กับ Cloud Computing

แนวคำตอบ: ควรเทียบ latency, bandwidth, processing power และ network resilience

61. 3. อธิบายขั้นตอนของ Object Detection ในภาพถนน

แนวคำตอบ: ควรพูดถึง bounding box, classification, confidence score และตัวอย่างรถบนถนน

62. 4. เปรียบเทียบ Two-stage detector กับ One-stage detector

แนวคำตอบ: ควรยก Fast/Faster R-CNN กับ YOLO และชี้ว่า YOLO เร็วกว่าเพราะ unified detection

63. 5. อธิบายการทำงานของ YOLO แบบ grid

แนวคำตอบ: ควรพูดถึงการแบ่งภาพเป็น grid cells, BBox, confidence, class probability, IoU และ final detection

64. 6. ทำไม NMS จึงเป็น bottleneck ในแยกเมืองที่รถเยอะ

แนวคำตอบ: ควรอธิบาย candidate boxes จำนวนมาก วัตถุซ้อนกัน และ latency เพิ่มขึ้น

65. 7. เปรียบเทียบ YOLO กับ RT-DETR ใน ITS

แนวคำตอบ: ควรเทียบ CNN/local vs Transformer/global, latency, hardware และ use case

66. 8. อธิบาย Precision, Recall, mAP และ IoU ด้วยตัวอย่าง ITS

แนวคำตอบ: ควรยก false alarm, miss target และความเข้มงวดของ mAP@50-95

67. 9. อธิบาย Vehicle ReID pipeline ตั้งแต่ภาพวิดีโอจนได้ผลลัพธ์

แนวคำตอบ: ควรเริ่มจาก detection, filtering, MTSC, inter-camera matching, MTMC และ result

68. 10. ทำไม City Scale ReID ต้องใช้ Spatial/Temporal Integration

แนวคำตอบ: ควรอธิบายว่าความคล้ายทางภาพอย่างเดียวอาจพลาด จึงต้องใช้ตำแหน่งกล้อง ทิศทาง และเวลาเดินทางช่วย